

Resume Analyzer and Job Matcher

CAI4002: Introduction to Artificial Intelligence | Group Project Proposal

Team Members: Martin Dang, Tuan Khang Phan

1. Project Overview

The Resume Analyzer and Job Matcher is an AI application that helps students and job seekers measure how well a resume fits a specific job description before they apply. The system reads an uploaded resume together with a target job posting, uses Natural Language Processing (NLP) to extract the meaningful information from each one (skills, technologies, education, and experience), and compares the two. It then returns a compatibility score, a breakdown of matching and missing qualifications, and concrete recommendations the applicant can act on to raise that score.

The motivation comes from the way modern hiring actually works. Most large employers screen applications with an Applicant Tracking System (ATS), and a large share of resumes are filtered out automatically before any human recruiter sees them, often because of missing keywords or weak alignment with the posting rather than a true lack of ability. Our tool gives applicants visibility into that hidden step: it behaves like a transparent, simulated ATS, so a user can understand why a resume might be rejected and fix it. At the same time, the project is a practical demonstration of how the AI and NLP techniques covered in this course apply to a real problem.

At a high level, the system follows this pipeline:

1. Extract text from the uploaded resume document (PDF or DOCX).
2. Parse the target job description using NLP techniques.
3. Identify the skills, technologies, education, and experience expressed in both documents.
4. Compute a similarity score that represents resume-to-job compatibility.
5. Detect missing or weakly supported skills and qualifications.
6. Generate ranked recommendations for improving the resume's alignment with the requirements.

2. Approach and Connection to Course Concepts

A central goal of this project is to apply the ideas from CAI4002 to a problem outside the Pacman and grid-world setting where we first learned them. Each stage of the pipeline above maps onto techniques from the course:

State and feature representation. Just as a search agent stores only the part of the world it needs (in our Pac-Man corners problem the state was the position plus four "visited" flags, not the whole board), the analyzer reduces a noisy resume to a compact, structured representation: a

set of extracted skills, an experience profile, and a numeric feature vector. Choosing the right representation is what makes the later comparison tractable.

Vector-space model and similarity. The match score is grounded in the vector-space ideas from the course. Each document is turned into a weighted term vector (TF-IDF) or a dense embedding produced by a transformer language model such as BERT, and resume-to-job compatibility is measured with cosine similarity between those vectors. This is the same "represent the object as a vector, then measure distance" pattern that underlies many of the methods we studied.

Constraint satisfaction. A job posting can be read as a set of constraints over a candidate. We model required qualifications as hard constraints (a specific degree, a mandatory certification) and preferred qualifications as soft constraints that raise the score when satisfied but do not disqualify the applicant. Framing the posting as a constraint-satisfaction problem makes the "matching versus missing" report a direct readout of which constraints are and are not satisfied.

Informed search for recommendations. The most course-relevant component is the recommendation engine. We treat resume improvement as a search problem: a state is a version of the resume, an action is a concrete edit (adding a skill the user genuinely has, rewording a bullet to surface a keyword, listing a missing certification), and the goal is any resume whose match score clears a target threshold. Because the space of possible edits is large, we rank candidate edits with a best-first, A*-style strategy guided by a heuristic that estimates how much "skill-gap distance" remains to the goal. This mirrors the admissible and consistent heuristics we designed for A* in the search project: the heuristic never overstates the work left, so the suggestions surfaced first are the ones expected to close the largest gap for the least effort.

Reasoning under uncertainty. Resume parsing is noisy, since a skill can be implied rather than stated and section boundaries are ambiguous. To handle this we attach a confidence to each extracted item using a probabilistic classifier (for example, a naive Bayes model over section text), which reflects the probabilistic reasoning introduced in the course and lets the final score communicate certainty rather than a single bare number.

An adversarial view. Finally, the ATS screening problem can be framed adversarially, much like the minimax setting from class: the applicant tries to maximize the chance of passing while the automated filter applies strict rules. Thinking of the screener as an adversary clarifies why surfacing and closing keyword gaps is the rational strategy for the applicant.

3. Background Research and Related Work

Matching resumes to job descriptions is an active research area, and several existing projects informed our design.

Lavi, Medentsiy, and Graus (2021) introduce conSultantBERT, a Siamese Sentence-BERT model fine-tuned on more than 270,000 real resume-vacancy pairs labeled by staffing consultants. They show that a fine-tuned, domain-specific embedding model clearly outperforms unsupervised baselines built on TF-IDF and off-the-shelf BERT embeddings, and that it handles multilingual content. Their work is the main evidence that learned embeddings, rather than raw keyword overlap, give the strongest matching signal, and it motivates our use of transformer embeddings alongside a simpler TF-IDF baseline.

Li et al. (2020) tackle a closely related pair of tasks, predicting a candidate's competence level and matching resumes to job descriptions, using context-aware transformer models evaluated on 6,492 real resumes. Their results reinforce that transformer models capture the contextual meaning of experience far better than bag-of-words methods, and their two-task framing (level prediction plus matching) influenced our decision to report both a numeric score and a qualitative skill breakdown.

On the applied side, two open-source projects shape our implementation plan. Pyresparser (Pathak, n.d.) is a Python resume parser built on spaCy and NLTK that extracts names, contact details, education, experience, and skills from PDF and DOCX files; it gives us a concrete, reusable starting point for the extraction stage. The AI-Powered Resume Matching System, or CV-Embed (Edirisinghe, n.d.), demonstrates an end-to-end embedding-based matcher and serves as a reference architecture for combining extraction, embedding, and similarity scoring into a single application.

Our project differs from these in its emphasis. Rather than only producing a match score, we focus on the explanatory, recommendation-oriented layer: telling the user exactly which qualifications are missing and, using the informed-search ranking described above, which edits will most improve their standing. That actionable feedback loop, framed explicitly in terms of the AI concepts from this course, is the contribution that distinguishes our system.

4. Data

The project does not require us to collect private resume data; we rely on publicly available datasets for development and testing, plus an authoritative skills taxonomy for normalization. All three sources below are free for academic use, which keeps the project reproducible and avoids the privacy concerns of scraping live applicant data.

- **Resume Dataset (Kaggle).** About 2,400 real resumes collected from livecareer.com, provided as both plain text and HTML and labeled into roughly 24 job categories (for example Information-Technology, HR, Finance, Engineering, and Healthcare). We use it to develop and test the extraction and category classification stages. <https://www.kaggle.com/datasets/snehaanbhawal/resume-dataset>
- **LinkedIn Job Postings 2023-2024 (Kaggle).** About 124,000 real job postings with the title, full description, location, and work type, plus companion files listing the skills and

industries tied to each posting. This supplies the job-description side of the matching problem and realistic skill vocabularies.
<https://www.kaggle.com/datasets/arshkon/linkedin-job-postings>

- **O*NET Database (U.S. Department of Labor).** A standardized occupational database covering more than 900 occupations with detailed skills, abilities, knowledge, and technology descriptors, released under a Creative Commons license. We use it as a controlled skills ontology to normalize differently worded skills (for example, mapping "JS" and "JavaScript" to one canonical skill) so the resume and job vectors are comparable. <https://www.onetcenter.org/database.html>

5. References

- arshkon. (n.d.). *LinkedIn job postings (2023-2024)* [Data set]. Kaggle.
<https://www.kaggle.com/datasets/arshkon/linkedin-job-postings>
- Edirisinghe, S. (n.d.). *AI-Powered Resume Matching System (CV-Embed)* [Computer software]. GitHub.
<https://github.com/Samitha-Edirisinghe/AI-Powered-Resume-Matching-System-CV-Embed>
- Lavi, D., Medentsiy, V., & Graus, D. (2021). *conSultantBERT: Fine-tuned Siamese Sentence-BERT for matching jobs and job seekers* (arXiv:2109.06501). arXiv.
<https://doi.org/10.48550/arXiv.2109.06501>
- Li, C., Fisher, E., Thomas, R., Pittard, S., Hertzberg, V., & Choi, J. D. (2020). Competence-level prediction and resume & job description matching using context-aware transformer models. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics. <https://doi.org/10.18653/v1/2020.emnlp-main.679>
- National Center for O*NET Development. (n.d.). *O*NET database* [Data set]. U.S. Department of Labor, Employment and Training Administration.
<https://www.onetcenter.org/database.html>
- Pathak, O. (n.d.). *pyresparser: A simple resume parser used for extracting information from resumes* [Computer software]. GitHub.
<https://github.com/OmkarPathak/pyresparser>
- snehaanbhawal. (n.d.). *Resume dataset* [Data set]. Kaggle.
<https://www.kaggle.com/datasets/snehaanbhawal/resume-dataset>